

Name: Kamran Gul

CMS: 023-25-0161

Section: C

OOP Lab: 13

Task 1

StudentCard — toString, equals, and roster logic

Scenario:

A campus **ID card** is identified by a **student id** (String, unique) and a **full name**. Printing a card should show something **readable**, not `StudentCard@1a2b3c`. Later, a **roster** (array of cards) must be **searched** and **filtered** by id prefix—typical **problem-solving** on arrays + `String` methods.

Core requirements

1. **Class `StudentCard`:**
 - Private fields: `studentId` (String), `fullName` (String).
 - Constructor: `StudentCard(String studentId, String fullName)` — if `studentId` is `null` or **empty** after `trim`, throw `new IllegalArgumentException(...)` with a short message.
 - `@Override public String toString()` — one line including **student id** and **full name** (e.g. `"StudentCard[id=S123, name=Ali Khan]"`).
 - `@Override public boolean equals(Object o)` — `instanceof` (or `getClass()`) pattern; equality **by `studentId` only** (document that **two** cards with same id but different `fullName` are still **equal** under this rule, or state if you require both fields—**one rule only**).
2. **Polymorphism in `Main` (core):**
 - Create two cards with the **same** `studentId` and different names; print `equals` result.
 - `Object ref = new StudentCard(...); System.out.println(ref);` — must use **your** `toString()`.

Problem-solving extensions (complete all)

3. **Instance method — prefix check:**
 - `public boolean studentIdStartsWith(String prefix)`
 - Return `true` iff `studentId` starts with `prefix`, treating `null` prefix as **not** starting (return `false`) or throw `IllegalArgumentException`—**document** your choice. Use `String` API (e.g. `startsWith`) or a careful loop.
4. **Static search — loop + `equals`:**
 - `public static StudentCard findByStudentId(StudentCard[] roster, String id)`
 - Return the **first** card in `roster` whose `studentId equals id` (`String.equals`), or `null` if none found or if `roster` is `null` (document: empty array returns `null`).
5. **In `Main` (extensions):**

- Build a `StudentCard[]` of at least **3** entries (include one id `"So10"`).
- Call `findByStudentId` for `"So10"` and print the result (or `null`).
- Pick one card and print `studentIdStartsWith("So")`.

Solution

```
class StudentCard {
    private String studentId;
    private String fullName;

    public StudentCard(String studentId, String fullName) {
        if (studentId == null || studentId.trim().equals("")) {
            throw new IllegalArgumentException("Invalid ID");
        }

        this.studentId = studentId;
        this.fullName = fullName;
    }

    public String toString() {
        return "ID: " + studentId + ", Name: " + fullName;
    }

    public boolean equals(Object o) {
        StudentCard other = (StudentCard) o;
        return this.studentId.equals(other.studentId);
    }

    public boolean studentIdStartsWith(String prefix) {
        if (prefix == null) {
            return false;
        }

        return studentId.startsWith(prefix);
    }

    public static StudentCard findByStudentId(StudentCard[] roster, String id) {
        if (roster == null) {
            return null;
        }

        for (int i = 0; i < roster.length; i++) {
            if (roster[i].studentId.equals(id)) {
                return roster[i];
            }
        }

        return null;
    }
}

class Main {
    public static void main(String[] args) {

        StudentCard s1 = new StudentCard("s001", "Ali");
        StudentCard s2 = new StudentCard("s001", "Ahmed");

        System.out.println(s1.equals(s2));

        Object obj = new StudentCard("s002", "Sara");
        System.out.println(obj);

        StudentCard[] cards = {
            new StudentCard("s010", "Usman"),
            new StudentCard("s011", "Hina"),
        };
    }
}
```

```

        new StudentCard("S012", "Bilal")
    };

    System.out.println(StudentCard.findByStudentId(cards, "S010"));
    System.out.println(cards[0].studentIdStartsWith("S0"));
}
}

```

Output

```

true
ID: S002, Name: Sara
ID: S010, Name: Usman
true

```

Task 2

AmountParser — parsing, batches, and failure counting

Scenario:

A form sends **amounts as text**. You **parse** and **reject** invalid or **negative** values. Real systems also process **many** fields at once: **sum** valid lines, or **count** how many lines fail without stopping the whole program.

Core requirements

- Class `AmountParser` (methods may be `static`):
 - `public static double parseNonNegativeAmount(String text)`
 - `trim`; if empty, throw `IllegalArgumentException("empty input")` (or similar).
 - `Double.parseDouble` inside `try`; `catch (NumberFormatException ex)` → throw `new IllegalArgumentException("not a number: " + text, ex)` (cause optional).
 - If parsed value `< 0`, throw `IllegalArgumentException("negative not allowed")`.
 - Return the `double` if valid.
- In `Main` (core):
 - `"120.5"` → print `120.5`.
 - `"abc"` in `try/catch` → print `getMessage()`.
 - `"-5"` → rejected (catch or message).

Problem-solving extensions (complete all)

- Strict batch sum:
 - `public static double sumNonNegativeAmounts(String[] lines)`
 - Loop over `lines`. For each element, call `parseNonNegativeAmount`. If any call throws, **rethrow** or wrap in `IllegalArgumentException` that includes the **array index** of the bad line (e.g. `"bad amount at index 2: ..."`).
 - If `lines` is `null`, throw `IllegalArgumentException` or return `0.0—document`.
- Soft failure count (no throw from helper):

- `public static int countInvalidAmountLines(String[] lines)`
- Return how many entries **would** fail `parseNonNegativeAmount` (catch **inside** the method per line, **do not** abort the whole loop on first failure). Treat `null` array as **0** invalid lines (document).

5. In `Main` (extensions):

- Call `sumNonNegativeAmounts` with `{"10", "20.5", "30"}` — print sum **60.5**.
- Call `countInvalidAmountLines` with `{"1", "x", "-2", "3"}` — expect **2** invalid ("x" and "-2").

Solution

```
class AmountParser {
    public static double parseNonNegativeAmount(String text) {
        if (text == null || text.trim().equals("")) {
            throw new IllegalArgumentException("empty input");
        }
        double value;
        try {
            value = Double.parseDouble(text);
        } catch (Exception e) {
            throw new IllegalArgumentException("not a number");
        }
        if (value < 0) {
            throw new IllegalArgumentException("negative not allowed");
        }
        return value;
    }

    public static double sumNonNegativeAmounts(String[] lines) {
        double sum = 0;
        for (int i = 0; i < lines.length; i++) {
            sum = sum + parseNonNegativeAmount(lines[i]);
        }
        return sum;
    }

    public static int countInvalidAmountLines(String[] lines) {
        int count = 0;
        for (int i = 0; i < lines.length; i++) {
            try {
                parseNonNegativeAmount(lines[i]);
            } catch (Exception e) {
                count++;
            }
        }
        return count;
    }
}

class Main {
    public static void main(String[] args) {
        System.out.println(
            AmountParser.parseNonNegativeAmount("120.5")
        );
    }
}
```

```

);
try {
    System.out.println(
        AmountParser.parseNonNegativeAmount("abc")
    );
} catch (Exception e) {
    System.out.println(e.getMessage());
}

try {
    System.out.println(
        AmountParser.parseNonNegativeAmount("-5")
    );
} catch (Exception e) {
    System.out.println(e.getMessage());
}

String[] a = {"10", "20.5", "30"};
System.out.println(
    AmountParser.sumNonNegativeAmounts(a)
);

String[] b = {"1", "x", "-2", "3"};
System.out.println(
    AmountParser.countInvalidAmountLines(b)
);
}
}

```

Output

```

120.5
not a number
negative not allowed
60.5
2

```

Task 3

BankAccount, InsufficientFundsException, and transfers

Scenario:

An account has a **balance**. **Withdraw** uses a **custom checked exception** when funds are insufficient. **Transfers** between accounts are a common **logic** exercise: move money only if the source can **withdraw** it, then **deposit** on the target.

Core requirements

1. Class `InsufficientFundsException` — `extends Exception`, constructor `super(message)`.
2. Class `BankAccount`:
 - `BankAccount(double initialBalance)` — `initialBalance < 0` → `IllegalArgumentException`.
 - `deposit(double amount)` — `amount <= 0` → `IllegalArgumentException`.
 - `withdraw(double amount)` throws `InsufficientFundsException` — `amount <= 0` → `IllegalArgumentException`; `amount > balance` → `InsufficientFundsException`; else subtract.
 - `getBalance()`, `@Override toString()`.
3. In `Main` (core):
 - `try/catch` on **failed** withdraw; **successful** withdraw and print balance.

Problem-solving extensions (complete all)

4. Transfer method:
 - `public void transferTo(BankAccount other, double amount)` throws `InsufficientFundsException`
 - `null other` → `IllegalArgumentException`.
 - `amount <= 0` → `IllegalArgumentException`.
 - **Otherwise:** perform `withdraw(amount)` on `this`, then `deposit(amount)` on `other`. (If `withdraw` throws, no deposit happens—do not partially transfer.)
5. Comparison helper:
 - `public static BankAccount accountWithHigherBalance(BankAccount x, BankAccount y)`
 - Return `x` if `x.getBalance() > y.getBalance()`, else `y` if `y` is higher; if **equal**, return `x` (document tie-break).
6. In `Main` (extensions):
 - Create **two** accounts; **deposit** different amounts.
 - `transferTo` from richer to poorer so both balances change; `println` both `toString()` results.
 - Call `accountWithHigherBalance` and print which account has the larger balance.

Solution

```
class InsufficientFundsException extends Exception {
    public InsufficientFundsException(String msg) {
        super(msg);
    }
}

class BankAccount {
    private double balance;

    public BankAccount(double initialBalance) {
        if (initialBalance < 0) {
            throw new IllegalArgumentException("Invalid balance");
        }

        balance = initialBalance;
    }

    public void deposit(double amount) {
        if (amount <= 0) {
            throw new IllegalArgumentException("Invalid deposit");
        }

        balance = balance + amount;
    }

    public void withdraw(double amount)
        throws InsufficientFundsException {

        if (amount <= 0) {
            throw new IllegalArgumentException("Invalid withdraw");
        }

        if (amount > balance) {
            throw new InsufficientFundsException("No funds");
        }

        balance = balance - amount;
    }

    public void transferTo(BankAccount other, double amount)
        throws InsufficientFundsException {

        withdraw(amount);
        other.deposit(amount);
    }

    public double getBalance() {
        return balance;
    }

    public static BankAccount accountWithHigherBalance(
        BankAccount x, BankAccount y) {

        if (x.getBalance() >= y.getBalance()) {
            return x;
        } else {
            return y;
        }
    }

    public String toString() {
        return "Balance = " + balance;
    }
}

class Main {
```

```

public static void main(String[] args) {

    BankAccount a1 = new BankAccount(200);
    BankAccount a2 = new BankAccount(100);

    try {
        a1.withdraw(50);
        System.out.println(a1);

        a2.withdraw(300);

    } catch (Exception e) {
        System.out.println(e.getMessage());
    }

    a1.deposit(100);
    a2.deposit(20);

    try {
        a1.transferTo(a2, 70);
    } catch (Exception e) {
        System.out.println(e.getMessage());
    }

    System.out.println(a1);
    System.out.println(a2);

    BankAccount high =
        BankAccount.accountWithHigherBalance(a1, a2);

    System.out.println(high);
}
}

```

Output

```

Balance = 150.0
No funds
Balance = 180.0
Balance = 190.0
Balance = 190.0

```